

Day 1 — Networking Foundations for Software and Platform Engineers

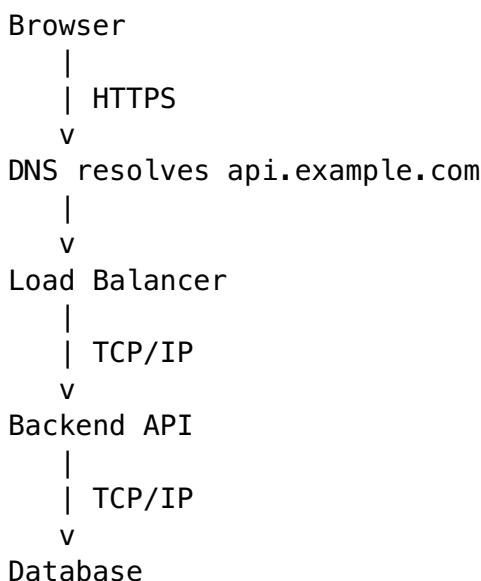
Beginner mental model

A network exists so that **one process running on one machine can exchange data with another process, potentially running on another machine thousands of kilometres away.**

For a backend/platform engineer, most networking can initially be understood through six questions:

1. **What application am I trying to reach?** → DNS / service discovery
2. **What IP address represents that destination?** → Layer 3
3. **How does my machine reach that IP?** → subnet, gateway, routing
4. **Which process on that machine should receive the traffic?** → TCP/UDP port
5. **Is communication permitted and secure?** → firewall, ACL, TLS
6. **What happens if packets are delayed, lost, or the service fails?** → TCP behaviour, retries, timeouts, load balancing

A browser talking to an API might conceptually look like:



Everything in today's networking foundation fits somewhere inside this flow.

1. What a computer network actually does

Host

A **host** is a device participating in a network.

Examples:

- laptop
- physical server

- virtual machine
- Kubernetes worker node
- database server
- smartphone

A host normally has at least one network interface and one or more IP addresses.

Client

A **client** initiates communication with another application.

Example:

Browser ----request----> Web API

Here the browser is the client.

But client/server are **roles**, not permanent machine identities.

For example:

Browser → Backend API → Database

The backend API is:

- a **server** from the browser's perspective
 - a **client** from the database's perspective
-

Server

A **server** is a program that waits for and responds to requests.

Examples:

nginx	listens on TCP 443
FastAPI service	listens on TCP 8000
PostgreSQL	listens on TCP 5432
DNS server	usually listens on UDP/TCP 53

A server is primarily **software**.

The physical/virtual computer running it is the host.

Network Interface / NIC

A **Network Interface Card/Controller — NIC** connects a host to a network.

Examples:

Laptop Ethernet interface
Laptop Wi-Fi interface
VM virtual network interface
Kubernetes node eth0
Cloud VM virtual NIC

One machine may have multiple interfaces.

Example:

Server
├ eth0 → application network
└ eth1 → management network

Each interface may have:

- MAC address
 - IP address
 - routing configuration
-

MAC address vs IP address

These are commonly confused.

MAC address

A MAC address identifies a network interface primarily within a **Layer-2 network**.

Example:

00:1A:2B:3C:4D:5E

Switches primarily use MAC addresses.

Think:

| MAC tells Ethernet which local interface should receive a frame.

IP address

An IP address provides **logical Layer-3 addressing**, allowing communication across different networks.

Example:

10.20.1.25

Routers use IP addresses and network prefixes.

Think:

| IP tells the network where the destination host/network is located.

Simplified distinction

MAC → local delivery on an Ethernet network

IP → communication across networks

An IP packet may travel across ten routers while its IP destination stays essentially the same.

The Ethernet MAC addresses can change at **every Layer-2 segment** along the path.

Port

An IP address identifies a host/interface.

A port identifies an application endpoint on that host.

Example:

10.10.1.20:443
 ^
 port

Some familiar defaults:

Application	Typical port
HTTP	80
HTTPS	443
DNS	53
SSH	22
PostgreSQL	5432
Redis	6379

This allows one host to run many services.

10.10.1.20
├── :22 SSH
├── :443 HTTPS
├── :5432 PostgreSQL
└── :6379 Redis

Socket

A **socket** is an operating-system abstraction used by applications for network communication.

Conceptually a network endpoint may be represented by:

protocol + IP address + port

For example:

TCP 10.10.1.20:443

A TCP connection can be uniquely identified by something similar to:

source IP
source port
destination IP
destination port
protocol

Example:

192.168.1.10:51024

↓

TCP

↓

203.0.113.10:443

The client normally receives an ephemeral source port such as 51024.

Packet

A **packet** is a Layer-3 unit of data, usually referring to an IP packet.

Conceptually:

```
+-----+
| IP Header      |
+-----+
| TCP/UDP data   |
+-----+
```

Ethernet frame

An Ethernet **frame** is the Layer-2 container used to transport data over an Ethernet network.

Conceptually:

```
+-----+
| Ethernet Header |
| src MAC / dst MAC |
+-----+
| IP Packet       |
+-----+
| Ethernet Trailer |
+-----+
```

So:

Ethernet frame
contains
↓
IP packet
contains
↓
TCP/UDP segment/datagram
contains
↓
application data

Connection

A **connection** is a logical communication relationship between endpoints.

TCP is connection-oriented.

For example:

Browser ===== API
TCP connection

UDP generally does not establish such a connection before sending data.

Protocol

A protocol is an agreed set of rules for communication.

Examples:

Ethernet	→	local network delivery
IP	→	addressing/routing
TCP	→	reliable transport
DNS	→	name resolution
HTTP	→	web request/response
TLS	→	secure communication

Without common protocols, systems would not understand each other's messages.

Link

A **link** connects network nodes.

Examples:

Laptop	----	Ethernet cable	----	Switch
Router	----	fibre connection	--	Router
VM	-----	virtual link	-----	virtual switch

Links can be:

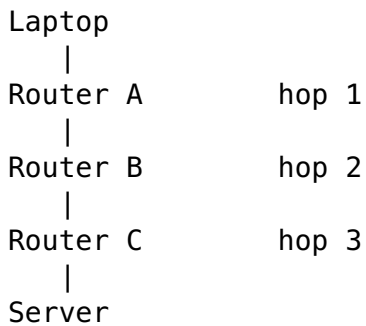
- physical

- wireless
 - virtual
-

Hop

A **hop** generally represents movement from one Layer-3 device/router to the next.

Example:



The Internet may involve many hops.

Bandwidth, throughput, latency, jitter and packet loss

These describe different network characteristics.

Bandwidth

Bandwidth is the theoretical or configured **capacity of a link**.

Example:

1 Gbps network interface

Think of bandwidth as the width of a highway.

Throughput

Throughput is the amount of useful data actually transferred per unit time.

You may have:

Link capacity: 1 Gbps
Actual throughput: 650 Mbps

because of:

- protocol overhead
- congestion
- packet loss
- application limitations
- server performance

So:

Bandwidth != throughput

Latency

Latency is how long communication takes.

Example:

Client → Server = 20 ms

Application latency may contain:

DNS time
+ connection time
+ TLS time
+ network travel
+ server processing
+ database time
+ return travel

Jitter

Jitter means **variation in latency**.

For example:

packet 1 → 10 ms
packet 2 → 11 ms
packet 3 → 70 ms
packet 4 → 9 ms

The connection might have acceptable average latency but terrible jitter.

Jitter matters particularly for:

- voice
 - video
 - real-time systems
-

Packet loss

Packet loss means some packets never reach their destination.

Example:

100 packets sent
97 packets received

3% packet loss

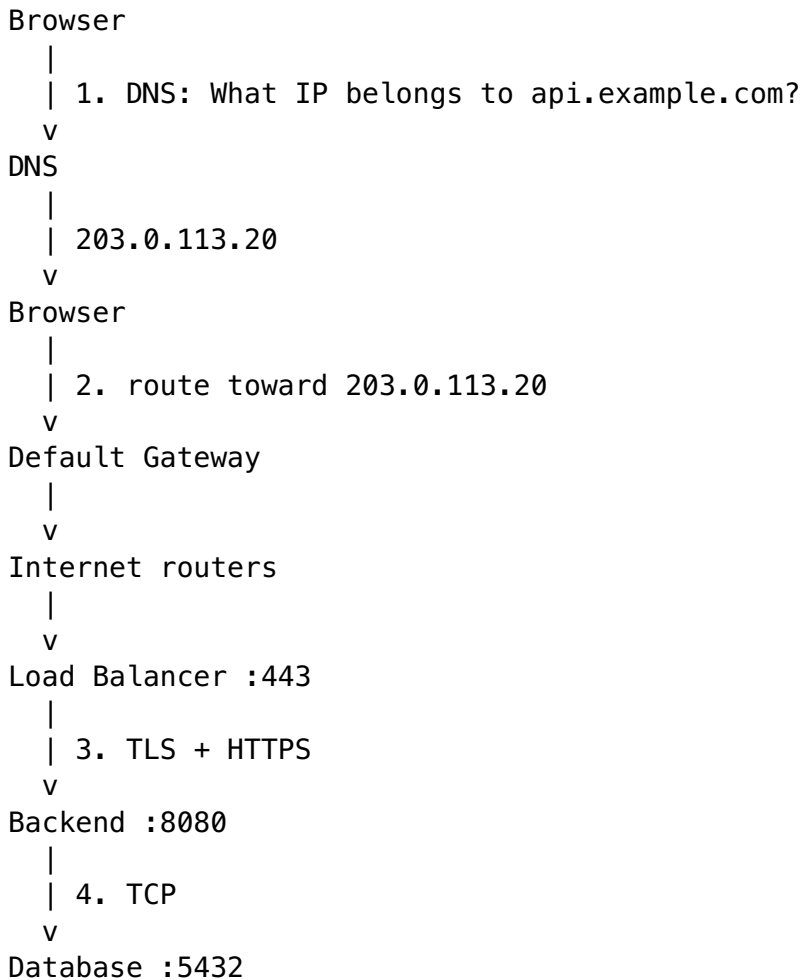
TCP may retransmit missing information, but this increases latency and reduces throughput.

Putting these concepts together

Suppose you open:

`https://api.example.com/users`

Very approximately:



The browser doesn't directly think about switches or Ethernet frames.

The OS and network infrastructure handle those layers.

2. OSI and TCP/IP models

The networking models are useful because they let us reason about **separate responsibilities**.

Do not treat them primarily as memorisation exercises.

A simplified model is enough for software engineering:

+-----+		
	Application	
	HTTP DNS TLS	
+-----+		
	Transport	
	TCP UDP	
+-----+		
	Network	
	IP ICMP	
+-----+		
	Data Link	
	Ethernet MAC ARP*	
+-----+		
	Physical	
	Cable Fibre Radio	
+-----+		

ARP sits at the boundary between Layer 2 and Layer 3 because it maps IPv4 addresses to Layer-2 MAC addresses. Different descriptions sometimes categorize it slightly differently; operationally, think of it as helping IP communicate over a local Ethernet network.

Physical layer

Concerned with transmission of raw signals.

Examples:

- copper cable
- fibre
- radio
- electrical/light signalling

Questions at this layer include:

Is the cable connected?
 Is the interface up?
 Is the wireless signal available?

Data Link layer

Provides local network communication.

Important concepts:

- Ethernet
- MAC addresses
- Ethernet frames
- switches
- VLANs

A switch normally operates primarily here.

Network layer

Provides logical addressing and routing between networks.

Important concepts:

- IP
 - subnet
 - router
 - routing table
 - ICMP
-

Transport layer

Provides communication between application processes.

Important protocols:

TCP
UDP

It introduces ports.

Application layer

Protocols applications understand directly.

Examples:

HTTP
DNS
SMTP
SSH

TLS is often discussed between application and transport because it secures application protocols carried over transport connections.

For practical backend reasoning:

HTTP over TLS over TCP over IP over Ethernet

Where common technologies belong

Technology	Main conceptual layer
Ethernet	Data Link
MAC	Data Link
ARP	L2/L3 boundary
IP	Network

Technology	Main conceptual layer
ICMP	Network
TCP	Transport
UDP	Transport
DNS	Application
HTTP	Application
TLS/HTTPS	Application/security over transport

Ethernet
↓ remove Ethernet header
IP
↓ remove IP header
TCP
↓ process TCP information
HTTP
↓
Application

This is **decapsulation**.

Header vs payload

A header contains control information.

Example IP header information:

source IP
destination IP
TTL
protocol

The **payload** is the data being carried.

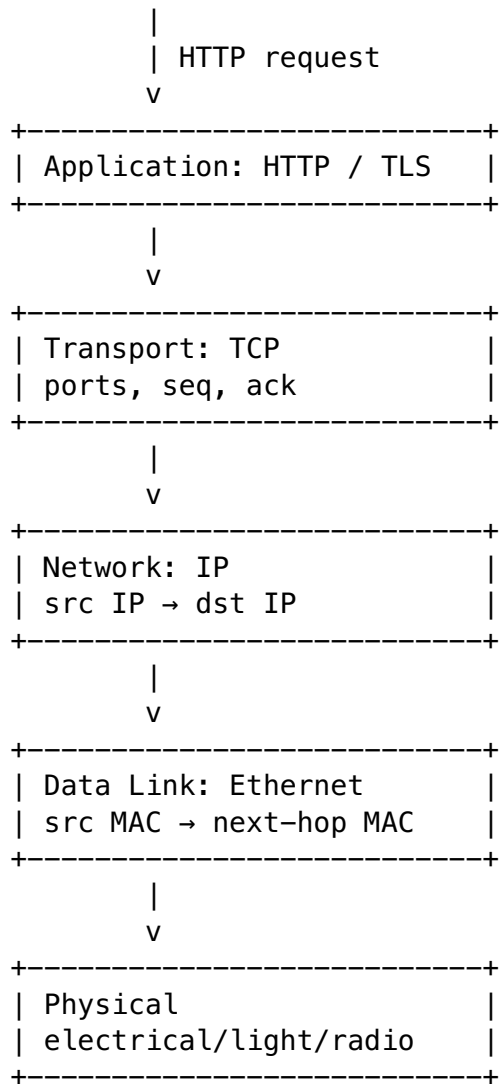
Conceptually:

Header	Payload
--------	---------

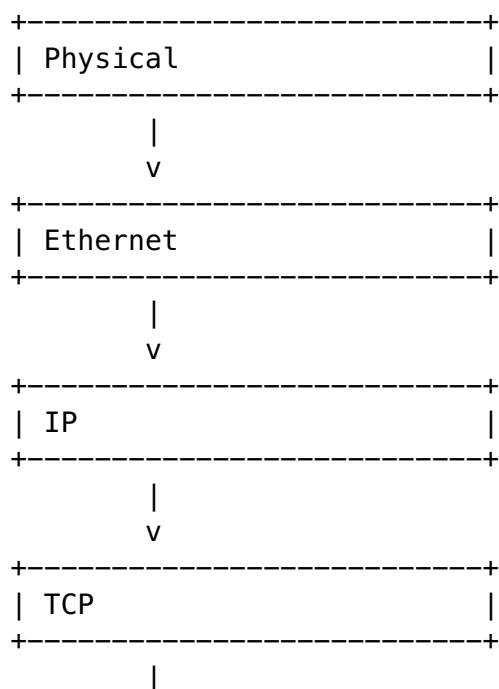
But one layer's entire message becomes another layer's payload.

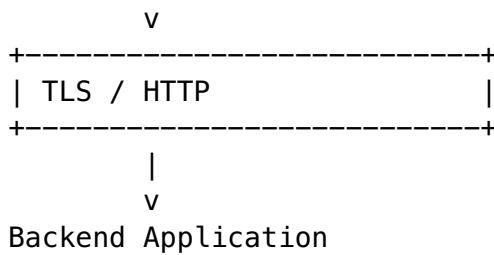
Diagram 1 — Application request across network layers

Browser/Application



NETWORK





3. Layer 2 networking

Layer 2 answers a basic question:

| How do devices communicate on the same local network?

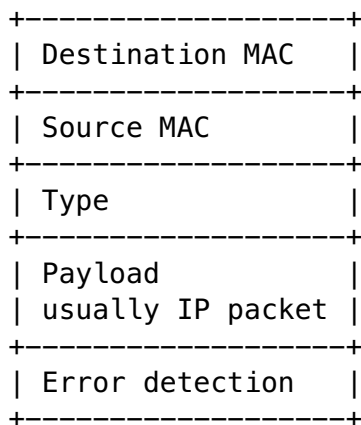
Ethernet

Ethernet is the dominant technology used for wired local networks and is also conceptually important in virtual/cloud environments.

Ethernet communicates using **frames** and **MAC addresses**.

Ethernet frame

Simplified:

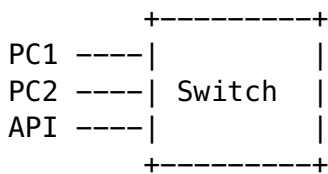


You do not normally need to memorize every field.

Switches

A switch connects devices within a Layer-2 network.

Example:



The switch learns which MAC addresses exist behind which ports.

MAC/CAM table

Suppose:

Switch port 1 → MAC A
Switch port 2 → MAC B
Switch port 3 → MAC C

The switch learns:

MAC	PORT

AA:AA:AA:AA	1
BB:BB:BB:BB	2
CC:CC:CC:CC	3

This table is often called:

- MAC address table
- forwarding table
- CAM table

Then a frame destined for MAC B is forwarded mainly toward port 2 rather than everywhere.

Unicast

Unicast means traffic from one sender to one specific destination.

Host A -----> Host B

Most application traffic is effectively unicast.

Broadcast

Broadcast means traffic intended for everyone in a Layer-2 broadcast domain.

Conceptually:

```

          +--> Host A
Sender ---+--> Host B
          +--> Host C

```

ARP requests are a classic example of local broadcast behaviour.

Broadcast domain

A broadcast domain is the set of devices that receive Layer-2 broadcasts.

Routers normally separate broadcast domains.

VLANs can also create separate broadcast domains.

ARP

Suppose:

Host A IP: 10.0.1.10

Host B IP: 10.0.1.20

Host A knows it wants 10.0.1.20, but Ethernet needs a destination MAC.

It needs to discover:

10.0.1.20 → which MAC address?

ARP provides this mapping.

Conceptually:

Host A broadcasts:

"Who has 10.0.1.20?"

Host B responds:

"10.0.1.20 is at BB:BB:BB:BB"

Host A can then construct an Ethernet frame addressed to Host B's MAC.

ARP cache

Repeating ARP resolution for every packet would be inefficient.

Hosts therefore temporarily cache mappings:

10.0.1.20 → BB:BB:BB:BB

10.0.1.1 → RR:RR:RR:RR

That is the ARP cache.

Entries eventually expire or change.

What happens when destination is on another subnet?

Suppose:

My machine:	10.0.1.10
Destination:	10.0.2.20
Default gateway:	10.0.1.1

Your machine does **not** ARP for the remote server's MAC.

Instead it finds:

10.0.1.1 → gateway MAC

and sends the Ethernet frame to the router.

Important distinction:

IP destination	= remote server
MAC destination	= next-hop router

That distinction is fundamental.

VLANs

Without segmentation, a large Layer-2 network becomes undesirable because:

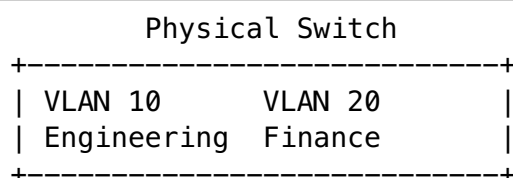
- broadcasts reach too many systems
- security boundaries are weak
- failures can spread
- administration becomes difficult

VLANs create multiple logical Layer-2 networks using the same switching infrastructure.

Example:

VLAN 10	→ Engineering
VLAN 20	→ Finance
VLAN 30	→ Servers
VLAN 40	→ Guest Wi-Fi

Conceptually:



Logically:

Engineering network	Finance network
separate L2 broadcast domains	

Traffic between VLANs normally requires Layer-3 routing.

Access ports

An **access port** normally belongs to one VLAN.

Example:

```
Laptop
|
| untagged Ethernet
|
Switch port
VLAN 10
```

The endpoint usually doesn't need to know about VLAN tagging.

Trunk ports

A trunk carries traffic for multiple VLANs.

Example:

```
Switch A
|
| VLAN 10
| VLAN 20
| VLAN 30
|
trunk
|
Switch B
```

802.1Q

802.1Q is the common Ethernet VLAN tagging mechanism.

Conceptually it adds information saying:

"This frame belongs to VLAN 20."

You don't need packet-bit details today.

VLAN ID

A VLAN ID identifies the VLAN.

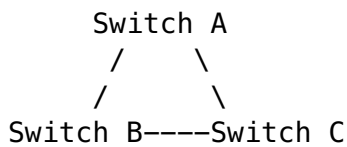
Examples:

```
VLAN 10
VLAN 20
VLAN 100
```

The standard VLAN ID field is 12 bits; not every value is normally available for ordinary VLAN use, but exact ranges aren't important to today's architectural understanding.

Layer-2 loops

Suppose switches are connected like:



This creates redundant paths.

Redundancy is good for resilience.

But Ethernet frames—especially broadcast traffic—can circulate through loops.

Unlike IP packets, classic Layer-2 Ethernet does not have the same router-hop TTL protection.

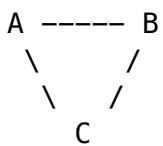
This can cause:

- broadcast storms
- duplicate frames
- MAC-table instability
- network collapse

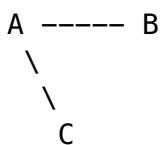
STP

Spanning Tree Protocol conceptually prevents Layer-2 loops by temporarily blocking some redundant paths while preserving them as backups.

Conceptually:



STP might logically disable one path:



If the active link fails, STP can allow an alternate link.

You do not need STP election details yet.

Office example

Imagine:

Corporate office switch

VLAN 10 → employee laptops
VLAN 20 → finance computers
VLAN 30 → printers
VLAN 40 → guest Wi-Fi

VLANs provide separation even though all systems may use the same physical switch infrastructure.

A router or Layer-3 switch handles communication between the VLANs subject to security policy.

4. Layer 3 networking

Layer 3 answers:

| How do packets move from one network to another?

The core protocol is **IP**.

IPv4 addressing

An IPv4 address contains 32 bits.

Human-readable example:

192.168.10.25

But the important thing architecturally is that it has:

network portion + host/interface portion

The prefix length tells us where the boundary is.

Example /24

192.168.10.25/24

The first 24 bits identify the network.

Conceptually:

Network:	192.168.10.0/24
Host:	.25

Devices in 192.168.10.0/24 can usually communicate directly at Layer 2, subject to VLAN/security design.

To reach another subnet, they normally use a router/default gateway.

Subnet

A subnet is an IP-address range representing a logical Layer-3 network.

Example:

10.10.1.0/24

You might assign it to:

application servers

And use another:

10.10.2.0/24

for:

databases

Subnet mask

The older-style subnet mask represents the same network/host division.

For /24:

255.255.255.0

CIDR notation is generally clearer for architecture discussions:

192.168.1.0/24

CIDR

CIDR means Classless Inter-Domain Routing.

Notation:

10.0.0.0/16

The /16 indicates how many bits belong to the network prefix.

A smaller prefix number describes a larger address block.

Examples:

/16 → large network
/24 → smaller network
/30 → tiny subnet
/32 → exactly one IPv4 address

Practical prefix intuition

/16

Example:

10.20.0.0/16

Contains roughly 65,536 IPv4 addresses in the mathematical block.

Common architectural use:

whole application environment / VPC range

Example:

VPC: 10.20.0.0/16

Then divide it:

10.20.1.0/24	web
10.20.2.0/24	application
10.20.3.0/24	databases

/24

Example:

10.20.10.0/24

Contains 256 addresses mathematically.

Historically, with traditional IPv4 subnet conventions, not all addresses were assignable to hosts because of network and broadcast addresses. Cloud providers may reserve additional addresses too.

Architecturally, think:

| convenient medium-sized subnet.

/30

Example:

10.20.30.0/30

Only four addresses mathematically.

Traditionally useful for point-to-point-style networks because the classic usable-host count is very small.

Modern networking may use other prefix sizes depending on platform and protocol.

/32

Example:

10.20.30.45/32

Represents exactly one IPv4 address.

You'll often see /32 used in:

- host routes
- firewall rules
- VPN routes
- BGP advertisements
- Kubernetes/network policies

Example:

Allow 10.20.30.45/32

means:

| allow only this specific address.

Default gateway

Suppose your server is:

10.10.1.20/24

and wants:

10.50.20.30

That destination is outside its local subnet.

The host sends the packet toward its **default gateway**, for example:

10.10.1.1

The gateway/router continues forwarding it.

Router

A router connects different IP networks.

Example:

Subnet A
10.1.0.0/24

Subnet B
10.2.0.0/24

Hosts --- Switch --- Router --- Switch --- Hosts

Routing table

A routing table tells the system:

| for this destination network, where should I send the packet?

Example:

Destination	Next hop

10.20.1.0/24	directly connected
10.30.0.0/16	10.20.1.1
172.16.0.0/12	10.20.1.2
0.0.0.0/0	10.20.1.254

Next hop

A next hop is the next router/interface toward the destination.

A router doesn't necessarily know the full path.

It primarily decides:

What is my next step?

Longest-prefix matching

Suppose a routing table contains:

10.0.0.0/8	→ Router A
10.20.0.0/16	→ Router B
10.20.10.0/24	→ Router C

Destination:

10.20.10.50

All three routes technically match.

The most specific match is:

10.20.10.0/24

Therefore the packet goes toward Router C.

This is **longest-prefix matching**.

Think:

| most specific route wins.

Default route

The default route is:

0.0.0.0/0

It means:

| use this route when no more specific route exists.

A typical server may have:

local subnet → directly connected
everything else → default gateway

Subnetting as architecture

Don't think of subnetting primarily as:

| How many hosts fit in /27?

Instead think:

| Which systems should belong to separate routing and security boundaries?

Example cloud design:

VPC 10.0.0.0/16

```
├── Public subnet
│   10.0.1.0/24
│   Load balancers
├── App subnet
│   10.0.10.0/24
│   Backend services
└── DB subnet
    10.0.20.0/24
    Databases
```

This enables different:

- routes
- firewall policies
- Internet exposure
- monitoring
- operational controls

Enterprise network connection

Example:

```
Employees
|
User VLAN/subnet
|
Firewall
|
Application subnet
|
Firewall
|
Database subnet
```

The IP architecture itself becomes part of the security architecture.

Kubernetes connection

Kubernetes introduces additional address spaces such as:

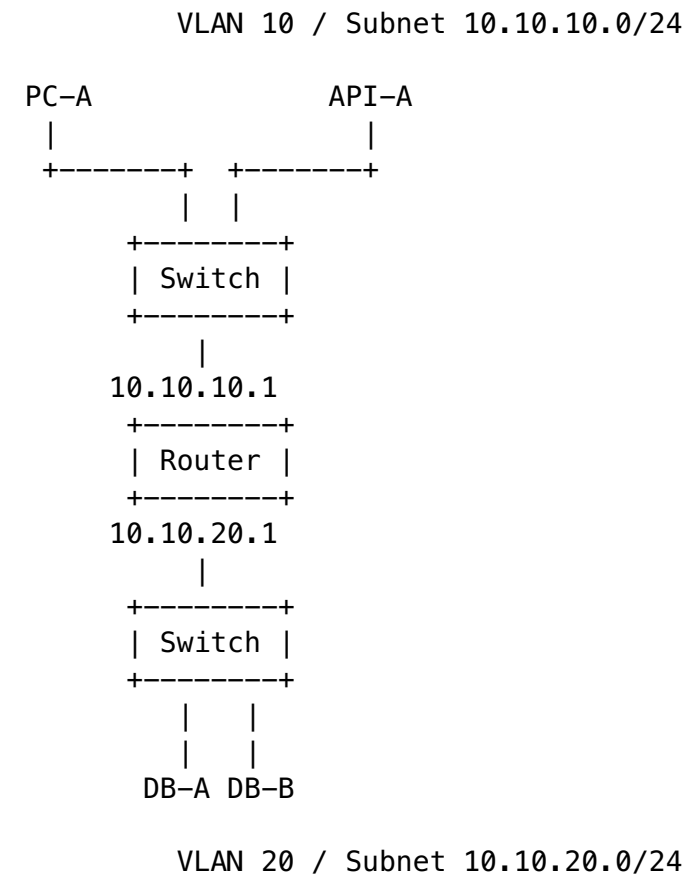
```
Node network
Pod network
Service network
```

Conceptually:

```
Node: 10.1.0.20
Pod:  10.244.1.7
Service: 10.96.20.10
```

The exact networking implementation varies by CNI, but IP addressing, routes, encapsulation, MTU and filtering still apply.

Diagram 2 — Switch, router and subnet topology



Within a subnet:

switch / Layer 2

Between subnets:

router / Layer 3

5. ICMP

ICMP stands for **Internet Control Message Protocol**.

IP itself delivers packets but needs mechanisms to report some network conditions.

ICMP helps communicate information such as:

destination unreachable
packet lifetime exceeded
reachability responses

Echo request and echo reply

ping commonly uses:

ICMP Echo Request

↓

destination

↓

ICMP Echo Reply

Conceptually, it asks:

| Can ICMP traffic reach this IP and return?

Important:

A failed ping does **not necessarily mean the server is down**.

ICMP may be blocked while HTTPS works perfectly.

Destination unreachable

A router/host may return an ICMP destination-unreachable message when delivery cannot occur.

Possible conceptual causes:

- network unreachable
- host unreachable
- certain protocol/port conditions

Exact behaviour varies.

Time exceeded

IP packets have a TTL — Time To Live.

Routers decrement it.

Example:

TTL = 3

Router A → TTL 2

Router B → TTL 1

Router C → TTL 0

When it expires, a router usually discards the packet and may send:

ICMP Time Exceeded

How traceroute works conceptually

Traceroute deliberately sends traffic with progressively larger TTL values.

For example:

TTL 1 → first router responds
TTL 2 → second router responds
TTL 3 → third router responds
...

This reveals the approximate Layer-3 path.

Different implementations may use different packet/protocol techniques, but TTL expiration is the central idea.

6. TCP

TCP is one of the most important protocols for backend engineers.

It provides a **reliable ordered byte stream** between applications.

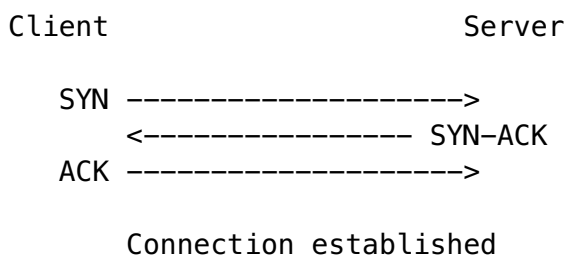
Examples:

- HTTPS
 - PostgreSQL connections
 - Redis connections
 - many service-to-service APIs
-

Connection oriented

Before normal application data exchange, TCP establishes a connection.

Conceptually:



This is the three-way handshake.

Why three messages?

At a high level, both sides confirm:

- the other side is reachable
 - they are ready
 - initial connection state can be synchronized
-

Sequence numbers

TCP data is tracked using sequence numbers.

Conceptually:

Bytes 1–1000
Bytes 1001–2000
Bytes 2001–3000

This lets TCP understand:

- what arrived
 - what is missing
 - how to reorder data
-

Acknowledgements

The receiver acknowledges received data.

Conceptually:

Sender		Receiver
bytes 1–1000	----->	
	<-----	ACK 1001

Meaning roughly:

| I have everything before byte 1001; send from there onward.

Actual TCP semantics are more sophisticated, but this mental model is sufficient now.

Retransmission

If data appears to be missing:

Sender:
packet/data A
packet/data B ← lost
packet/data C

TCP can retransmit the missing information.

This is one reason TCP appears reliable to applications.

Ordering

IP packets can potentially arrive out of order.

TCP reassembles them before presenting the byte stream to the application.

Example network arrival:

1
3
2

Application sees logically:

1
2
3

Flow control

Flow control protects the **receiver**.

Imagine:

Fast sender → slow receiver

The receiver has finite buffers.

TCP allows the receiver to communicate how much additional data it can accept.

Think:

| Don't overwhelm the receiver.

Congestion control

Congestion control protects the **network**.

Imagine thousands of senders transmitting at maximum speed through a limited-capacity router.

TCP adapts its sending rate based on signals such as loss and network conditions.

Think:

Flow control → protect receiver
Congestion control → protect network

This distinction is important.

TCP connection close

A connection is normally closed through an exchange allowing each direction to finish gracefully.

Conceptually:

Connection opened
used
unused for 5 minutes
proxy closes it

This matters when applications attempt to reuse old pooled connections.

Connection pooling

Opening a new connection can cost:

TCP handshake
+
TLS handshake
+
authentication/setup

Applications therefore reuse connections.

Example:

```
Backend
|
Connection Pool
├── DB connection 1
├── DB connection 2
├── DB connection 3
└── DB connection 4
```

Benefits:

- reduced setup latency
- controlled number of connections
- better performance

But an oversized pool can overload a database.

Retries

Retries can help when a failure is temporary.

But careless retries can make outages worse.

Example:

1000 requests fail
each retries 5 times

→ potentially 5000 extra attempts

Retries need:

- bounded attempts
- timeouts
- backoff
- jitter
- idempotency awareness

Networking and resilience engineering are tightly connected.

Broken connections

Connections can break because of:

- server restart
- NAT state expiration
- load balancer idle timeout
- link failure
- process crash
- firewall state change

Applications must assume connections are not permanent.

7. UDP

UDP is a simpler transport protocol.

It is connectionless from the protocol's perspective.

No TCP-style handshake is required.

Sender ----- datagram -----> Receiver

UDP does not inherently guarantee:

- delivery
 - ordering
 - retransmission
 - duplicate suppression
-

Why use UDP?

Because sometimes the application values:

- low overhead
- low latency
- application-controlled reliability
- multicast/broadcast capabilities in some environments

Examples include:

- DNS queries
- voice/video traffic
- some game traffic
- telemetry
- modern transports such as QUIC use UDP as their substrate and implement richer capabilities above it

TCP vs UDP

Characteristic	TCP	UDP
Connection setup	Yes	No TCP-style setup
Reliable delivery	Built in	No
Ordering	Built in	No
Retransmission	Built in	No
Flow control	Yes	No
Congestion control	Yes	Not inherently
Overhead	Higher	Lower
Application complexity	Often simpler	App may handle reliability
Typical examples	HTTPS, DB	DNS, media, QUIC substrate

Do not reduce the choice to:

TCP = slow
UDP = fast

That is too simplistic.

The application's protocol requirements matter more.

8. MTU, MSS and fragmentation

MTU

MTU means **Maximum Transmission Unit**.

It describes the maximum Layer-3 packet size that can be carried over a link without fragmentation at that interface.

Common Ethernet MTU:

1500 bytes

Conceptually:

| A road has a maximum vehicle size.

Why MTU exists

Networking technologies have limits on the frame/packet sizes they efficiently support.

If an IP packet is too large for a path, something must happen:

- fragmentation in cases where it is allowed
- source sends smaller packets
- packet is dropped and path-MTU mechanisms help adaptation

IPv4 and IPv6 differ in fragmentation behaviour, but the engineering lesson is the same:

| packet size must fit the path.

Fragmentation concept

Suppose:

Packet = 2000 bytes

Link MTU = 1500

Under appropriate IPv4 conditions, fragmentation might divide it into multiple IP fragments.

Fragmentation is generally undesirable because it adds complexity and inefficiency.

Modern systems prefer avoiding it when possible.

Path MTU

The path between two systems may contain many links:

```
Host
|
1500 MTU
|
Router
|
1450 MTU
|
VPN
|
1400 MTU
|
Destination
```

The effective usable path MTU is limited by the smallest relevant link.

This is the **Path MTU** concept.

TCP MSS

MSS means **Maximum Segment Size**.

It concerns how much TCP payload should be placed into a TCP segment.

Conceptually:

MTU

- └ IP header
 - └ TCP header
 - └ TCP payload = constrained by MSS
-

MSS helps TCP avoid creating packets too large for the path.

MTU mismatch problems

Symptoms may be strange.

For example:

small requests work
large requests fail

or:

TCP handshake works
TLS handshake stalls

or:

ping with small packets works
file upload hangs

These symptoms can happen when larger packets are dropped somewhere in the path.

VPN connection

VPN encapsulation adds additional headers.

Suppose physical path supports:

1500 bytes

VPN adds:

60 bytes of encapsulation

The inner packet may need to be smaller.

Therefore VPN interfaces may use something like:

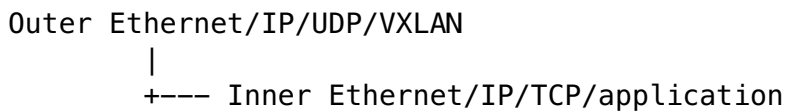
MTU 1440

instead of 1500.

Overlays and VXLAN

Overlay networking adds another network header around the original packet.

Conceptually:



More headers mean less room for application payload unless the underlying network supports a larger MTU.

This is highly relevant in:

- Kubernetes
 - SDN
 - cloud networking
 - VXLAN data-centre networks
-

9. DNS

Humans prefer names:

api.example.com

Networks route using IP addresses.

DNS translates names into information applications can use.

Think:

DNS = distributed naming system

Domain names

Example:

api.example.com

Conceptually:

.	root
com	top-level domain
example	registered domain
api	hostname/subdomain

DNS resolver

Your application normally asks a local resolver rather than contacting Internet DNS hierarchy directly.

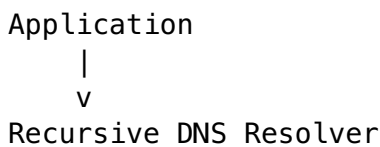
Possible resolver location:

- operating system
 - local network
 - enterprise DNS
 - ISP
 - public DNS service
-

Recursive resolver

A recursive resolver performs lookup work on behalf of the client.

Conceptually:

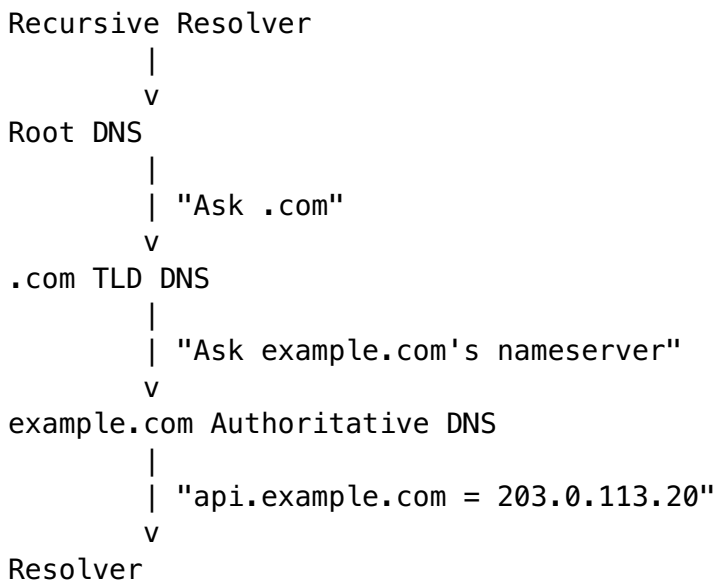


The resolver may answer from cache.

If not, it discovers the answer.

DNS hierarchy

Simplified lookup:



Root servers

Root DNS servers know where TLD DNS systems such as:

.com
.org
.net
.in

can be found.

They don't normally directly store the IP of every website.

TLD servers

For .com, the relevant TLD servers can direct queries toward authoritative DNS servers for:

example.com

Authoritative DNS

The authoritative DNS service contains DNS records controlled for that domain/zone.

Example:

api.example.com A 203.0.113.20

A record

Maps a name to an IPv4 address.

api.example.com
↓
203.0.113.20

AAAA record

Maps a name to an IPv6 address.

Example:

api.example.com
↓
2001:db8::20

CNAME

CNAME provides an alias.

Example:

api.example.com

↓

my-load-balancer.provider.example

Then that target is resolved further.

TTL

DNS records include a Time To Live.

Example:

TTL = 300 seconds

A resolver may cache that result for up to the relevant caching lifetime.

Benefits:

- fewer DNS queries
 - lower lookup latency
 - reduced authoritative-server load
-

What happens for api.example.com?

Suppose your application calls:

https://api.example.com/orders

The sequence is roughly:

Application

|

| resolve api.example.com

v

OS / local resolver

|

| cache miss

v

Recursive DNS

|

| if needed

+--> Root

+--> .com TLD

+--> example.com authoritative

|

v

203.0.113.20

|

v

Application opens TCP connection
to 203.0.113.20:443

DNS does not carry the application request itself.

It generally helps discover where to send the request.

Production DNS failures

Incorrect DNS

Suppose:

`api.example.com`

accidentally points to:

`old server`

All application code may be healthy, yet users fail because name resolution gives the wrong destination.

Stale cache

Suppose the service moves:

Old: `203.0.113.10`

New: `203.0.113.20`

Resolvers may temporarily continue using the old cached record.

This is one reason DNS migrations require planning.

DNS outage

If applications cannot resolve dependencies:

`service-a → service-b.internal`

connections can fail before TCP is even attempted.

This is why DNS becomes critical infrastructure.

TTL too long

Suppose:

TTL = 24 hours

Changing the IP may take a long time to propagate through caches.

Failover becomes slower.

TTL too short

Suppose:

TTL = 1 second

Resolvers must query far more frequently.

Possible consequences:

- increased DNS load
- more dependency on resolver availability
- more lookup overhead

The right TTL depends on stability and failover needs.

10. DHCP

DHCP means **Dynamic Host Configuration Protocol**.

Without DHCP, administrators might need to manually configure every device with:

- IP address
- subnet mask/prefix
- default gateway
- DNS resolver

That doesn't scale.

DHCP provides network configuration dynamically

A joining device can obtain information such as:

IP address
subnet mask
default gateway
DNS servers
lease duration

Lease

An IP allocation is often granted for a limited period.

Example:

Laptop gets:

10.10.20.51

Lease:
8 hours

The client can renew the lease.

This allows addresses to be reused.

Enterprise role

Imagine an office with 5,000 laptops.

Administrators don't want to manually assign:

5000 different IP configurations

Instead DHCP can dynamically allocate addresses per VLAN/site.

Example:

Engineering VLAN
DHCP pool:
10.20.10.50 – 10.20.10.220

When an employee joins the network, their laptop automatically gets configuration.

Servers, infrastructure and special systems may still use static/reserved addressing.

11. NAT

NAT means **Network Address Translation**.

It rewrites IP addressing information while traffic crosses a network boundary.

Private IP addresses

Common IPv4 private ranges include:

10.0.0.0/8
172.16.0.0/12
192.168.0.0/16

These aren't globally routed on the public Internet.

Example internal addresses:

10.20.1.10
10.20.1.11

Public IP address

A public address is globally routable on the Internet, subject to routing and policy.

A company may have thousands of private systems but only some public-facing addresses.

Basic NAT

Suppose:

Laptop: 10.0.0.20

accesses:

Internet server: 203.0.113.100

The NAT device may translate:

Source:

10.0.0.20

↓

198.51.100.10

The Internet sees the public address.

SNAT

Source NAT changes the **source address**.

Example:

Before:

src = 10.0.1.20

dst = 203.0.113.100

After SNAT:

src = 198.51.100.10

dst = 203.0.113.100

Typical use:

private systems → Internet

DNAT

Destination NAT changes the **destination address**.

Example:

Public:

198.51.100.20:443

translated toward:

10.0.20.50:443

Historically this has been common for publishing internal services.

Modern cloud load balancers may implement related concepts differently internally.

Port address translation

Many internal connections can share one public IP because the NAT device can distinguish them using transport ports.

Example:

```
10.0.0.10:50001 --\  
10.0.0.11:51025 -----> 198.51.100.10  
10.0.0.12:55015 --/
```

The NAT device maintains translation state.

This is often called PAT or NAT overload.

Enterprise Internet connectivity

Typical:

```
Employee private network  
  |  
  v  
Corporate firewall/NAT  
  |  
  | public IP  
  v  
Internet
```

Cloud networking

Typical private cloud subnet:

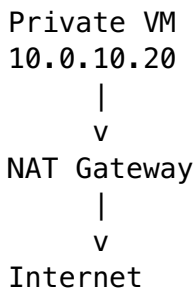
```
10.0.10.0/24
```

Servers may not have public IP addresses.

They reach the Internet through:

```
NAT Gateway
```

Conceptually:



Inbound Internet connections generally cannot simply initiate arbitrary connections back through the NAT mapping.

Kubernetes and NAT

Kubernetes implementations frequently use source/destination address translation in various paths, for example around:

- Service addressing
- node egress
- external traffic

Exact behaviour depends on CNI, kube-proxy mode, service type and platform.

The important point:

| NAT is still translation of network identity between communication domains.

Firewalls and NAT

A firewall and NAT are different concepts.

NAT → rewrite addressing
Firewall → permit/deny communication

A device may perform both.

12. Firewalls and ACLs

Networking asks:

| Can I reach the destination?

Security asks:

| Should I be allowed to?

Firewall

A network firewall applies security policy to traffic.

Rules often evaluate:

source
destination
protocol
source/destination port
direction
connection state

Example:

ALLOW
source: application subnet
destination: database subnet
protocol: TCP
port: 5432

Stateful firewall

A stateful firewall remembers connections.

Suppose:

Client → Server:443

is permitted.

The firewall recognizes corresponding return traffic as part of the same connection.

Conceptually:

Outbound connection allowed
 |
 v
Firewall remembers state
 |
 v
Return packets accepted

Stateless filtering

A stateless filter evaluates each packet mainly based on the rule set without maintaining full connection state in the same way.

Therefore rules may need to account explicitly for traffic in both directions.

ACL

Access Control List is a set of permit/deny rules.

Example:

```
ALLOW TCP
FROM 10.10.1.0/24
TO 10.10.2.10/32
PORT 443
```

ACLs exist in many environments:

- routers
 - switches
 - firewalls
 - cloud subnet controls
 - applications/network appliances
-

Source

Who sent the packet?

Example:

10.20.1.25

Destination

Where is it going?

10.30.1.50

Protocol

Example:

TCP
UDP
ICMP

Port

Example:

TCP destination port 443

Ingress

Traffic entering a system/network/security boundary.

Example:

Internet
|
v ingress
Firewall

Egress

Traffic leaving a system/network/security boundary.

Private subnet
|
v egress
Internet

Network segmentation

Rather than creating one flat network:

everything <--> everything

create zones:

User network
Web network
Application network
Database network
Management network

and control communication between them.

This reduces blast radius.

Security zones

A firewall may conceptualize networks as:

Internet
DMZ
Application
Database
Management

Policies define which zones can communicate.

DMZ

A DMZ is a network segment intended to host systems that must interact with less trusted networks, commonly the Internet, while separating them from trusted internal systems.

Conceptually:

```
Internet
|
Firewall
|
DMZ
|
Firewall/policy
|
Internal network
```

Modern cloud architectures may implement this pattern through public/private subnets and layered controls rather than using the exact classic term.

North-south traffic

Traffic entering or leaving a data centre/cloud/application environment.

Example:

```
Internet
|
v
Application
```

or:

Application → Internet

East-west traffic

Traffic between internal systems.

Example:

```
Service A → Service B
Service B → Database
Pod A → Pod B
```

Modern microservices generate very large amounts of east-west traffic.

13. TLS from a networking/application perspective

HTTP by itself does not encrypt communication.

Conceptually:

```
HTTP
Client ----- Server
      plaintext
```

Someone capable of observing the traffic could potentially inspect it.

HTTPS

HTTPS means HTTP protected by TLS.

HTTP

↓

TLS

↓

TCP

↓

IP

The communication becomes encrypted in transit.

TLS purposes

TLS provides important security properties including:

Encryption

Someone observing the connection should not be able to read normal application contents.

Integrity

Tampering can be detected.

Authentication

The client can verify the identity associated with the server's certificate, subject to the certificate trust model.

Certificates

A TLS certificate associates a cryptographic identity with information such as a hostname.

Example:

api.example.com

The client verifies that the certificate is valid for the requested hostname and chains to a trusted certificate authority, among other checks.

High-level TLS handshake

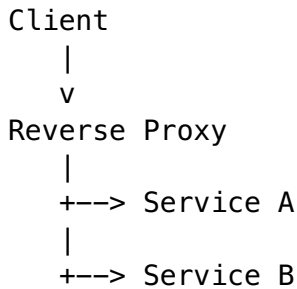
Conceptually:

Common enterprise use:

- controlled Internet access
 - filtering
 - auditing
 - caching
-

Reverse proxy

A reverse proxy represents **servers**.



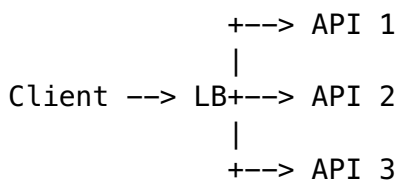
Clients connect to the proxy without needing to know backend details.

Common examples/roles:

- nginx
 - Envoy
 - ingress gateways
 - API gateways
-

Load balancer

A load balancer distributes traffic among multiple service instances.



Reasons:

- horizontal scaling
 - availability
 - maintenance
 - traffic distribution
-

Layer 4 load balancing

Layer 4 primarily makes forwarding decisions based on transport/network information such as:

source/destination IP
TCP/UDP
ports

Conceptually:

TCP :443
|
v
L4 Load Balancer
|
+--> server 1
+--> server 2

It does not need to deeply understand HTTP semantics.

Layer 7 load balancing

Layer 7 understands application protocol information.

For HTTP, it may route based on:

hostname
URL path
HTTP headers
cookies

Example:

/api/users → User Service
/api/orders → Order Service
/api/payments → Payment Service

That is extremely common in microservice architectures.

Health checks

A load balancer should stop routing requests to unhealthy servers.

Example:

LB
|
+--> Backend A ✓
+--> Backend B ✓
+--> Backend C ✗

Traffic goes only to healthy instances.

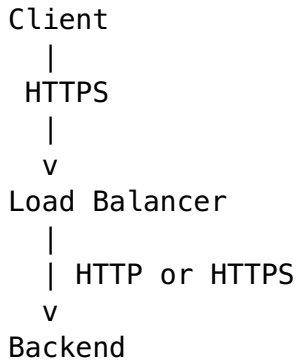
Good health checking must distinguish:

- process alive

- application actually ready
-

TLS termination

Instead of every backend handling public TLS, a load balancer/proxy can terminate it.



Many architectures still encrypt internal traffic too.

Session persistence

Ideally backends are stateless.

But sometimes requests from the same user need to reach the same backend.

Session persistence may use:

- cookie
- source information
- other affinity mechanisms

Conceptually:

User A → Server 1
User A → Server 1
User A → Server 1

This can reduce flexibility, so stateless application design is usually preferred where possible.

Connection distribution

A load balancer needs an algorithm or policy to select backends.

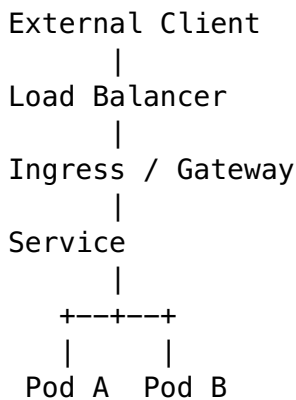
Conceptually:

round robin
least connections
hash-based approaches
weighted distribution

Detailed algorithms can come later.

Kubernetes connection

Kubernetes commonly introduces:



Different components may perform:

- L4 load balancing
- L7 routing
- TLS termination
- service discovery
- health checks

But the fundamental networking concepts remain the same.

15. QoS basics

QoS means **Quality of Service**.

Networks have finite capacity.

Suppose a company link is:

100 Mbps

and simultaneously handles:

software backup
VoIP call
video meeting
ERP transaction
large download

If everything is treated identically during congestion, real-time traffic may suffer badly.

Classification

Traffic is identified into categories.

Example:

Voice
Video
Business application
General web
Backup

Prioritisation

Some traffic can receive preferential treatment.

Example:

voice packets

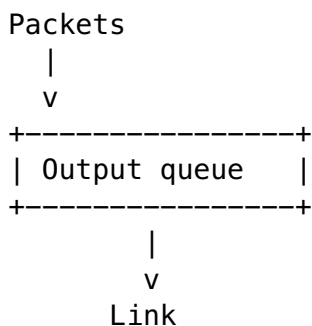
may be prioritized over:

nightly backup traffic

because voice is highly sensitive to delay and jitter.

Queuing

When packets arrive faster than a link can transmit them, they wait in queues.

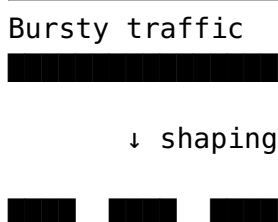


QoS determines how different classes share those queues.

Shaping

Traffic shaping smooths or limits traffic to a configured rate by buffering excess traffic.

Conceptually:



Think:

| delay some traffic so transmission stays near an intended rate.

Policing

Policing enforces a traffic rate more strictly.

Traffic above the permitted rate may be:

- dropped
- remarked

Think:

Shaping → usually buffers/delays excess

Policing → often drops or marks excess

Why this matters

Example priorities:

Highest sensitivity:

VoIP

Medium:

interactive business system

Lower urgency:

large backup job

QoS doesn't magically create bandwidth.

It controls how congestion is handled.

16. LAN, WLAN, WAN, Internet and intranet

LAN

Local Area Network.

Covers a limited environment such as:

- office floor
- building
- data-centre segment

Example:

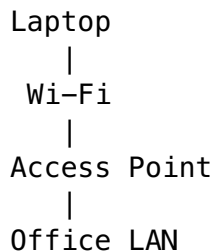
PCs → switches → local router

WLAN

Wireless LAN.

A LAN using wireless technologies such as Wi-Fi.

Example:

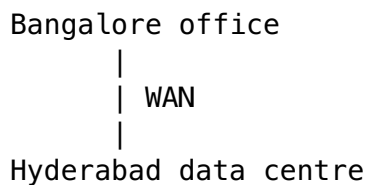


WAN

Wide Area Network.

Connects geographically separated networks.

Example:



WAN connectivity might use:

- carrier networks
 - MPLS
 - SD-WAN
 - leased circuits
 - VPN over Internet
-

Internet

The global network of interconnected IP networks.

It is public shared infrastructure, though access paths and services are controlled by network operators.

Intranet

An intranet is a private organization network/service environment using Internet-style technologies.

Example:

employees only:
<https://internal.company.example>

It might exist across:

- offices
 - data centres
 - cloud
 - VPN
-

Enterprise connectivity

Office users

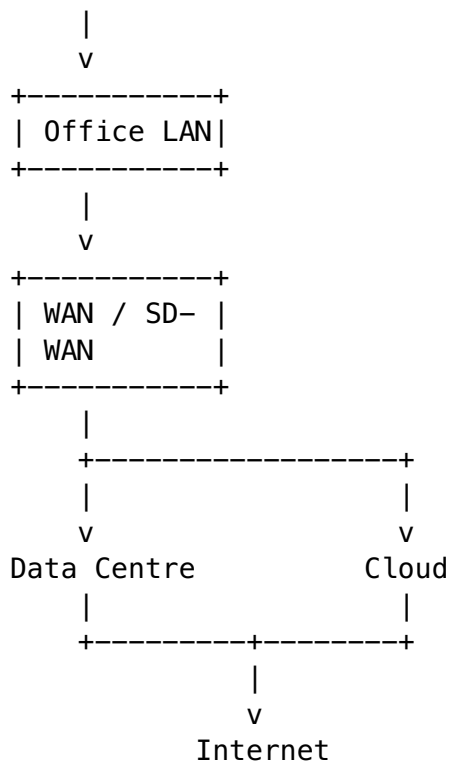
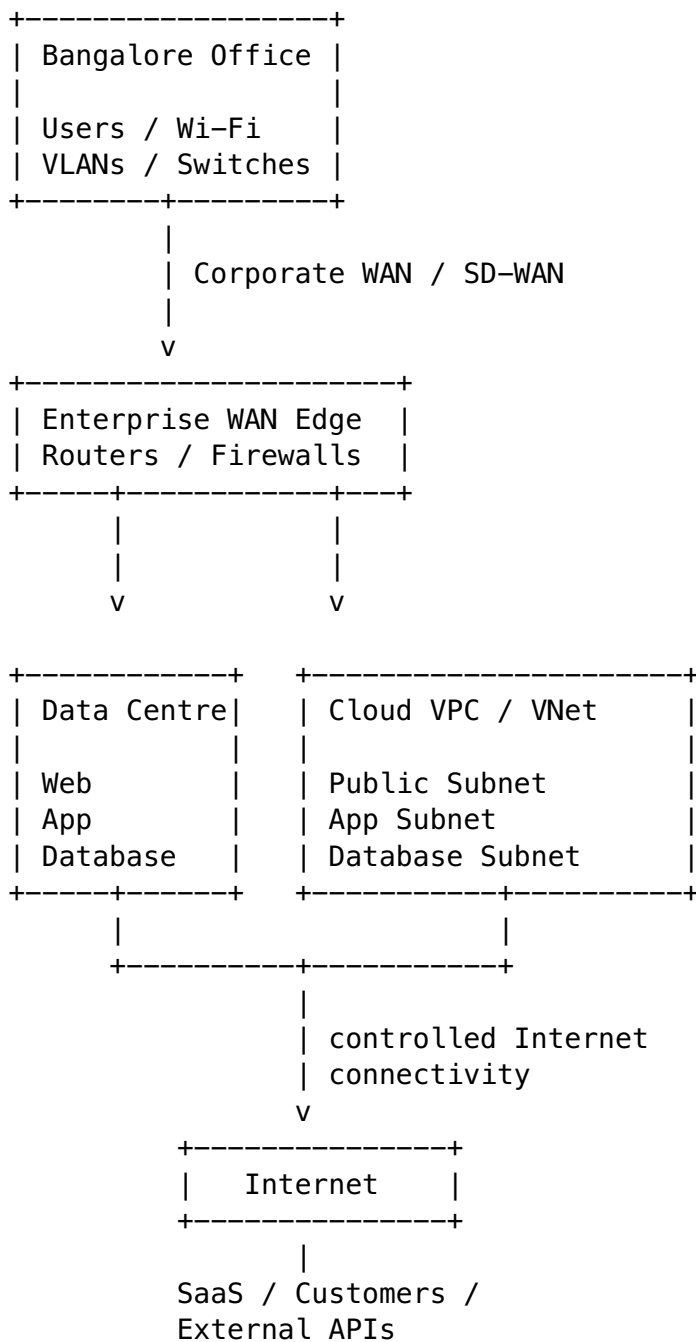


Diagram 3 — Enterprise office → data centre → cloud



17. Physical networking vs cloud networking

Cloud networking does not replace networking fundamentals.

It mostly gives us **software-defined versions of familiar network concepts**.

Physical networking

You may have:

Physical switches
Physical routers
Firewall appliances
Copper/fibre links
Physical racks

Example:

Server
|
Ethernet
|
Switch
|
Router
|
Firewall
|
Internet

Cloud networking

You work with abstractions:

VPC / VNet
Subnets
Route tables
Security groups
Network ACLs
Internet Gateway
NAT Gateway
Cloud Load Balancer

You may never touch a physical cable.

But underneath, physical network infrastructure still exists.

Mapping physical ideas to cloud ideas

Physical concept	Cloud abstraction
Network	VPC / VNet
IP subnet	Cloud subnet
Router routing table	Route table
Firewall policy	Security group / firewall
Router ACL	Network ACL
Internet edge router	Internet gateway abstraction
NAT appliance	NAT gateway
Hardware LB	Cloud load balancer

Physical concept	Cloud abstraction
Physical links	Provider-managed network fabric

These are not always exact one-to-one equivalents, but they are useful mental mappings.

VPC / VNet

A logically isolated cloud network.

Example:

VPC
10.0.0.0/16

Within it:

Public subnet
App subnet
DB subnet

Route table

Still answers:

| For this destination prefix, where should traffic go?

Example:

10.0.0.0/16 → local
0.0.0.0/0 → Internet Gateway

or private subnet:

0.0.0.0/0 → NAT Gateway

Same Layer-3 principle.

Security group

Conceptually a stateful firewall associated with resources/interfaces in many cloud platforms.

Example:

Application SG:

Inbound:

TCP 443 from LB security group

Database SG:

Inbound:

TCP 5432 from Application SG

Exact terminology and semantics differ by cloud provider.

Network ACL

Typically subnet-oriented packet filtering in cloud environments.

The details vary between AWS, Azure and other platforms, but the architectural concept remains access control.

Internet/NAT gateway

Conceptually:

Internet gateway

→ public Internet connectivity

NAT gateway

→ private systems initiating Internet access

Specific routing and addressing depend on cloud provider architecture.

Underlying principles that do not change

Whether physical or cloud:

Addresses still matter

Subnets still matter

Routes still matter

TCP ports still matter

DNS still matters

MTU still matters

Firewalls still matter

Latency still matters

Packet loss still matters

Cloud removes much physical management.

It does not remove networking.

18. Network failure reasoning

For platform engineers, the best habit is not memorizing twenty commands.

Instead ask:

| At which layer could communication be failing?

Use a layered approach.

Step 1 — Application/process

Question:

| Is the server process actually running and listening where expected?

Example:

Expected:

API listens :8080

Actual:

API listens :8000

Everything in the network could work perfectly, but connections to 8080 fail.

Possible problems:

- application crashed
 - service listening on wrong port
 - application bound only to localhost
 - connection limit exhausted
 - service overloaded
-

Step 2 — DNS

Question:

| Does the application name resolve to the intended IP?

Example:

api.example.com

↓

Wrong IP

Possible issues:

- bad record
- resolver unavailable
- stale cache
- split-DNS problem

- service-discovery failure
-

Step 3 — Transport

Question:

| Can TCP/UDP communication be established correctly?

Potential TCP problems:

- connection timeout
 - connection reset
 - service not listening
 - backlog/resource exhaustion
 - idle connection expired
-

Step 4 — Security

Question:

| Is something intentionally blocking the traffic?

Potential components:

Host firewall
Cloud security group
Network ACL
Enterprise firewall
Kubernetes network policy
Service mesh policy

Example:

App subnet → DB:5432

route works
DB listening

but firewall rule allows only :3306

Result: communication fails.

Step 5 — Routing

Question:

| Is there a valid path to the destination and back?

Example failure:

Request route exists:

A → B

Return route missing:

B -X-> A

Connections can still fail.

Networking requires reachability in the needed directions.

Potential problems:

- missing route
 - incorrect next hop
 - wrong subnet
 - overlapping networks
 - asymmetric path interacting badly with stateful devices
-

Step 6 — Layer 2

Inside a local network ask:

| Can the host reach its next-hop neighbour?

Potential problems:

- switch failure
 - VLAN mismatch
 - ARP problem
 - Layer-2 loop
 - interface down
-

Step 7 — Physical/link

Potential problems:

- cable failure
- fibre failure
- wireless interference
- interface failure
- carrier outage

Cloud users rarely see the physical details directly, but provider fabric failures still ultimately originate somewhere below your virtual abstractions.

Specific failure examples

Host failure

Client → Server X

Server X crashes.

Possible symptoms:

- connection refused
- timeout
- connection reset
- load balancer health check failure

Load balancing can reduce impact.

Link failure

A -----X----- B

If there is no redundant path:

communication stops

With routing redundancy:

A → Router 2 → B

traffic may reroute.

Switch failure

Devices connected only through that switch may become unreachable.

Example:

```
Servers
|
Switch X fails
|
Network
```

Data centres therefore often use redundant network paths.

Router failure

Traffic between networks may stop until:

- alternate route takes over
- routing reconverges
- failed component is restored

DNS failure

Interesting symptom:

`https://api.example.com → fails`

while direct IP connectivity may still be possible.

That strongly suggests investigating name resolution.

Firewall blocking

Typical conceptual sequence:

DNS works
route exists
destination healthy

BUT

security policy denies TCP :443

TCP connection cannot be established.

Routing problem

Example:

`10.20.0.0/16`

accidentally routed to the wrong next hop.

Symptoms may include:

- timeout
 - unexpected path
 - only certain subnets unreachable
-

Packet loss

TCP retransmits lost data.

Symptoms:

slow requests
high latency
reduced throughput
connection instability

Severe loss can produce timeouts.

High latency

Networking may technically work but performance suffers.

Example:

API → database
5 ms normally

becomes

250 ms

If a request makes 10 sequential database calls:

$10 \times 250\text{ms} \approx 2.5 \text{ seconds}$

Networking directly affects application architecture.

MTU problem

Often looks confusing:

Small packets work
Large requests fail
VPN path behaves strangely
TLS sometimes stalls

This is why MTU is worth understanding even for application engineers.

Connection timeout

A TCP connection attempt is sent, but establishment does not finish before the client's deadline.

Possible causes:

routing failure
firewall silently dropping
server unreachable
network outage
service overloaded

Compare with connection refused:

host reachable
but nothing accepts that TCP port

The precise behaviour depends on network and OS conditions, but this distinction is useful.

Layered troubleshooting mental model

Instead of randomly testing things, reason:

-
1. Application
Is the process alive and listening?

↓
 2. DNS
Did the name resolve correctly?

↓
 3. Transport
Can TCP/UDP reach the expected port?

↓
 4. Security
Are firewall/policies permitting it?

↓
 5. Routing
Does a valid forward + return path exist?

↓
 6. Layer 2
Can the local next hop be reached?

↓
 7. Physical / provider infrastructure
Is the underlying connectivity healthy?
-

You may investigate in a different order depending on evidence, but this framework prevents random troubleshooting.

End-to-end example: Browser → API → backend → database

Now combine everything.

Suppose you enter:

`https://shop.example.com/orders`

Step 1 — DNS

Browser needs the server's IP.

shop.example.com
|
v
DNS resolver
|
v
203.0.113.50

Step 2 — Determine route

Laptop:

192.168.1.50/24

Destination:

203.0.113.50

Not local.

Therefore:

send via default gateway
192.168.1.1

Step 3 — ARP

Laptop needs the default gateway's MAC address.

Who has 192.168.1.1?

Router responds.

Step 4 — Ethernet frame

Laptop creates something conceptually like:

Ethernet:

src MAC = laptop
dst MAC = router

IP:

src IP = laptop IP/NAT-side identity initially
dst IP = 203.0.113.50

TCP:

src port = 51024
dst port = 443

Step 5 — NAT

Corporate/home router may translate:

192.168.1.50:51024
↓
198.51.100.25:62001

Step 6 — Internet routing

Routers forward according to IP routing tables:

Router A
↓
Router B
↓
Router C
↓
Destination network

At every Layer-2 link, Ethernet framing may change.

The destination IP remains the logical end destination unless translation occurs.

Step 7 — Load balancer

Traffic reaches:

203.0.113.50:443

which may be a load balancer.

Client → LB +--> API Pod 1
 +--> API Pod 2
 +--> API Pod 3

Step 8 — TLS

Client verifies the certificate for:

shop.example.com

A secure session is established.

Step 9 — HTTP

Browser sends:

GET /orders

Step 10 — Backend

The API may need an internal service:

```
Order API
  |
  v
Inventory Service
```

Again:

```
DNS/service discovery
IP routing
TCP
firewall/security
HTTP/gRPC
```

The same principles repeat internally.

Step 11 — Database

Backend might connect:

```
Database:
10.20.30.50:5432
```

Possible path:

```
Backend subnet
  |
route
  |
firewall
  |
Database subnet
```

TCP provides reliable communication.

The entire stack

```
User
|
v
Browser
|
| DNS
v
Resolve hostname
|
v
IP destination
|
| subnet/routing
v
Default gateway / routers
|
| firewall / NAT
v
Internet / WAN
|
v
Load Balancer
|
| TLS + HTTP
v
Backend Service
|
| service discovery
| routing
| TCP
v
Database
```

The most important conceptual distinctions

Keep these very clear.

Concept A	Concept B	Difference
MAC	IP	Local L2 identity vs routable L3 address
IP	Port	Host/interface addressing vs application endpoint
Packet	Frame	L3 unit vs L2 container
Switch	Router	Same-L2-network forwarding vs between-network routing
Bandwidth	Throughput	Capacity vs actual transfer rate
Latency	Jitter	Delay vs variation in delay
DNS	Routing	Find IP from name vs move packets toward IP
NAT	Firewall	Address translation vs access decision
TCP	HTTP	Transport vs application protocol

Concept A	Concept B	Difference
TCP	TLS	Reliable transport vs secure communication
Flow control	Congestion control	Protect receiver vs protect network
L4 LB	L7 LB	Connection-level vs application-aware routing
Ingress	Egress	Entering vs leaving
East-west	North-south	Internal traffic vs entering/leaving environment
Private IP	Public IP	Internal addressing vs globally routable addressing
Subnet	VLAN	Layer-3 IP boundary vs Layer-2 logical segment

Networking and Kubernetes

When you later learn Kubernetes networking, avoid thinking it is a completely new networking world.

You will encounter:

Pod IP
Node IP
Service IP
CNI
Ingress
NetworkPolicy
kube-proxy
VXLAN
Service mesh
LoadBalancer

But underneath are today's concepts:

IP addresses
routes
subnets
TCP/UDP
DNS
NAT
MTU
encapsulation
firewall policies
load balancing

Example:

```
Client
|
Cloud Load Balancer
|
Ingress
|
Kubernetes Service
|
Pod
|
Database
```

Every arrow still requires some combination of:

```
name resolution
addressing
routing
transport
security
```

Networking and cloud

Similarly:

```
AWS VPC
Azure VNet
GCP VPC
```

may look like cloud-specific concepts.

But the mental model remains:

```
address space
↓
subnets
↓
routes
↓
security policies
↓
gateways
↓
load balancers
↓
applications
```

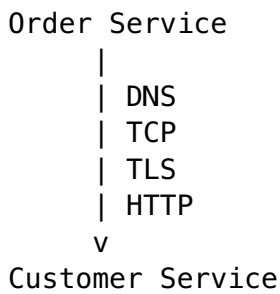
The provider automates much of the physical infrastructure underneath.

Networking and microservices

A monolith might make a function call:

```
orders.get_customer()
```

A microservice architecture may turn that into:



This adds failure modes:

DNS failure
connection failure
timeout
packet loss
TLS failure
service overload
load-balancer issue
routing issue

Therefore networking knowledge becomes more important as systems become distributed.

Final coherent mental model

Think of communication as a sequence of questions.

Application wants:

`https://api.example.com/users`

1. DNS

What IP is `api.example.com`?

→ `203.0.113.20`

2. Layer 3

Is that IP local?

No.

Which route?

→ default gateway

3. Layer 2

What MAC belongs to my next hop?

ARP determines gateway MAC.

4. Physical/local transport

Ethernet frame travels to switch/router.

5. Routing

Routers repeatedly choose the next hop using destination prefixes and longest-prefix matching.

6. Security

Firewalls/ACLs determine whether communication is allowed.

7. Transport

TCP connects to destination port 443.

8. TLS

Server proves identity.
Encrypted session established.

9. Application

HTTP request:

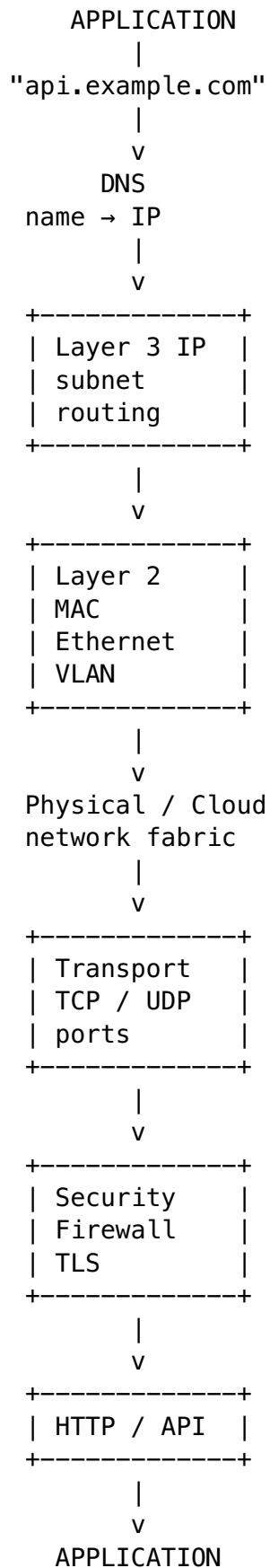
GET /users

10. Backend dependency

The server may repeat the process:

Backend
|
| DNS + route + firewall + TCP
v
Database :5432

The essential model is therefore:



The single most useful principle to retain from Day 1 is:

Applications communicate through names and ports, DNS turns names into addresses, Layer 3 finds the path between networks, Layer 2 handles local delivery

to the next hop, TCP/UDP transports application data, firewalls determine what is permitted, and TLS protects communication.

Once this model is solid, topics such as **VPCs, Kubernetes networking, service meshes, ingress controllers, VPNs, BGP, cloud load balancers and enterprise network automation** become extensions of the same fundamentals rather than isolated technologies.